Modifying LeanCoP Options on the Fly

(aka "the PAAR/AITP paper")

Introduction

In LeanCoP, small changes to proof-search control have large practical effects [8, 7]. These variants are studied in a PyCoP-style environment with explicit proof actions, including `cut`, `comp(J)`, and `Reset(n)`.

Fixed strategy schedules are already central to leanCoP-style prover engineering [8], and recent work has treated schedule design itself as a learning problem [5, 6]. The present experiments stay at the hand-designed level, but use that scheduling perspective throughout.

Strategies

The following strategies are used in the schedules.

FirstAction
The first legal action is selected [3, 8].

LeanCopCut
LeanCoP-style inclusive restricted backtracking is used: successful reductions and extensions are cut, and `comp(7)` is used for completeness [8].

ReductionCut
Inclusive cut is applied only on reductions [1].

HeuristicReductionCut
Forward actions are ordered heuristically, preferring reductions and then shorter clauses; cut is applied only on reductions [3, 1].

SubgoalReordering
A stricter categorical order is used: reductions, then shorter extensions, then shorter starts [3].

EntropyCut
The baseline order is used, but `cut` is added only when the softmax entropy of heuristic action scores is low [9].

AdaptiveReset
Dead-end backtracks are replaced by explicit resets to a larger depth bound, using a simple search-state heuristic [2, 4].

SmartCut
Entropy-based cut is combined with adaptive reset [9, 2, 4].

FewAlternativesCut
The default depth-first order and `comp(7)` are used, but `cut` is applied only when the current choice point is narrow [8, 1].

GoalDistanceCut
LeanCoP-style cut is used, but forward actions are ordered by an estimate of proximity to closure [3].

DepthPhasedCut
Reductions are always cut, while extensions are cut only near the top of the iterative-deepening tree; `comp(10)` is used [8].

Schedules

P1 60s
Maximises total solved problems.

(1)	`AdaptiveReset`	435 ms
(2)	`SubgoalReordering`	1917 ms
(3)	`LeanCopCut`	44 495 ms
(4)	`SmartCut`	1228 ms
(5)	`FirstAction`	10 295 ms
(6)	`GoalDistanceCut`	1626 ms

P2 60s
Maximises solved problems not solved by LeanCoP.

(1)	`SmartCut`	863 ms
(2)	`EntropyCut`	37 776 ms
(3)	`GoalDistanceCut`	9123 ms
(4)	`HeuristicReductionCut`	581 ms
(5)	`AdaptiveReset`	4588 ms
(6)	`ReductionCut`	2793 ms
(7)	`SubgoalReordering`	1917 ms
(8)	`FewAlternativesCut`	1749 ms

P1 120s
Maximises total solved problems.

(1)	`SubgoalReordering`	3705 ms
(2)	`AdaptiveReset`	4588 ms
(3)	`LeanCopCut`	95 539 ms
(4)	`HeuristicReductionCut`	581 ms
(5)	`EntropyCut`	13 619 ms
(6)	`GoalDistanceCut`	1584 ms
(7)	`DepthPhasedCut`	339 ms

Setting	Solved	Unsolved by `LeanCopCut`	Unsolved by `FirstAction`	Unsolved by both
`FirstAction` (60s)	2049	174	—	—
`LeanCopCut` (60s)	2164	—	289	—
`P1 60s`	2291	171	323	64
`P2 60s`	2140	247	174	101
`FirstAction` (120s)	2134	166	—	—
`LeanCopCut` (120s)	2304	—	336	—
`P1 120s`	2449	173	387	74
`P2 120s`	2292	256	227	113

Table 1: Solved-problem counts for the controls and the four portfolios.

P2 120s

Maximises solved problems not solved by LeanCoP.

(1) `SmartCut`	863 ms
(2) `SubgoalReordering`	13 410 ms
(3) `DepthPhasedCut`	31 596 ms
(4) `HeuristicReductionCut`	581 ms
(5) `GoalDistanceCut`	1584 ms
(6) `EntropyCut`	43 111 ms
(7) `AdaptiveReset`	4588 ms
(8) `ReductionCut`	23 446 ms

Time allocations are chosen by exact maximum-coverage optimisation, and slot order is chosen by exact optimisation of serial runtime.

Results

The violin plots in Fig. 1 use the same score as in Mangla et al. [6]. With total budget T and consumed time t, the score is

$$s = \begin{cases} 0.5 + \frac{T-t}{2T} & \text{on success,} \\ \frac{T-t}{2T} & \text{otherwise.} \end{cases}$$

Each row in Table 1 is evaluated at its own global budget. A portfolio is emulated slot by slot from the measured runtimes: a slot succeeds only if its strategy solves within that slot's cap; otherwise its consumed time is charged and evaluation proceeds. *Unsolved by `LeanCopCut`*, *Unsolved by `FirstAction`*, and *Unsolved by both* are set differences against the budget-matched control solve sets.

References

[1] Michael Färber. *A Curiously Effective Backtracking Strategy for Connection Tableaux*. Jan. 16, 2024. arXiv: 2106.13722 [cs]. URL: http://arxiv.org/abs/2106.13722 (visited on 02/02/2026). Pre-published.

[2] Sean B. Holden. "Machine Learning for Automated Theorem Proving: Learning to Solve SAT and QSAT". In: *Foundations and Trends® in Machine Learning* 14.6 (2021), pp. 807–989. ISSN: 1935-8237, 1935-8245.

[3] Reinhold Letz and Gernot Stenz. "Model Elimination and Connection Tableau Procedures". In: *Handbook of Automated Reasoning*. Elsevier, 2001, pp. 2015–2114. ISBN: 978-0-444-50813-3.

[4] Chunxiao Li et al. "A Reinforcement Learning Based Reset Policy for CDCL SAT Solvers". In: *CoRR* abs/2404.03753 (Apr. 19, 2024). arXiv: 2404.03753 [cs.LO]. Pre-published.

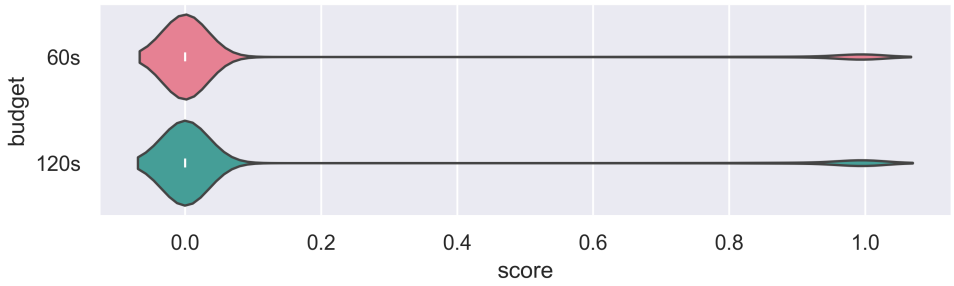
[5] C. Mangla, S.B. Holden, and L.C. Paulson. "Bayesian Ranking for Strategy Scheduling in Automated Theorem Provers". In: *International Joint Conference on Automated Reasoning (IJCAR)*. 2022, pp. 559–577.

[6] Chaitanya Mangla. "Machine Learning for Theorem Prover Strategies". PhD thesis. University of Cambridge, 2025.

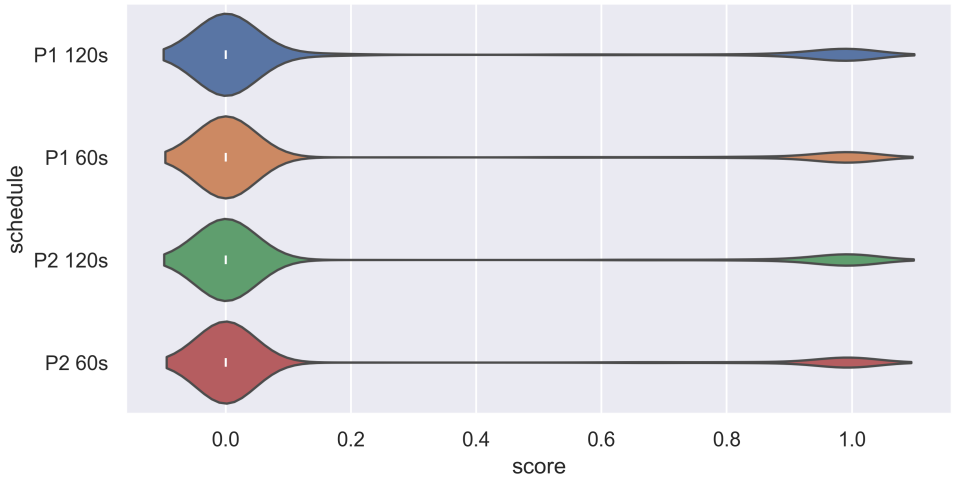
[7] Jens Otten. "20 Years of leanCoP — an Overview of the Provers". In: *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi (AReCCa@TABLEAUX 2023)*. Vol. 3613. CEUR Workshop Proceedings. 2023.

[8] Jens Otten. "Restricting Backtracking in Connection Calculi". In: *AI Communications* 23.2–3 (2010), pp. 159–182. ISSN: 09217126.

[9] Zsolt Zombori, Josef Urban, and Miroslav Olšák. "The Role of Entropy in Guiding a Connection Prover". In: *Automated Reasoning with Analytic Tableaux and Related Methods*. Ed. by Anupam Das and Sara Negri. Vol. 12842. Cham: Springer International Publishing, 2021, pp. 218–235. ISBN: 978-3-030-86058-5 978-3-030-86059-2.



(a) Control violin plots.



(b) Test violin plots.

Figure 1: Violin plots of the score distributions. The shorter subfigure shows the controls; the longer subfigure shows the test schedules.